School of Future Tech

Case Study Report

on

PolicyPick: Health Insurance Comparison Tool

by

Nimish Bordiya

150096725027

Index

Sr. No.	Topic
1	Introduction to the Case Study
2	Problem Statement / Case Background (Abstract)
3	Problem Statement / Case Study Design
4	Methods & Algorithms / Technology Applied
5	Implementation Details and Snapshots
6	Results and Conclusion
7	References

1. Introduction to the Case Study

My project is called PolicyPick. It is a website where people can compare different health insurance plans easily. I built this website using HTML, CSS, and JavaScript as part of my Semester II case study.

The idea came from a simple problem — when someone wants to buy health insurance, there are too many plans available and it is very hard to understand which one is better. The language used by insurance companies is very complicated, and most people get confused.

So I made a website where you can see all the important details of 6 different insurance plans on one screen, compare them side by side, and also filter them by your budget. The website is simple to use and shows everything in plain language without any confusing terms.

The website has 3 pages:

- Compare Page — shows all plans in a table so you can compare them easily.
- All Plans Page — shows each plan as a card with key details.
- Plan Details Page — shows complete information about one specific plan.

2. Problem Statement / Case Background (Abstract)

Background

Right now, if someone wants to compare health insurance plans, they have to visit many different websites, read long documents, and still not understand what they are getting. This wastes a lot of time and confuses people.

The main problems are:

- Insurance plans use difficult words like "waiting period", "co-payment", "sub-limit" etc. which common people do not understand.
- There is no simple way to see two or three plans next to each other and compare them.
- The premium price is usually shown yearly, so people do not know how much they are paying every month.
- Extra benefits like dental cover, vision cover, maternity cover are not shown clearly.

Abstract

To solve these problems, I built PolicyPick — a simple 3-page website using HTML, CSS, and JavaScript. It shows 6 health insurance plans with all important details. Users can filter

plans by their monthly budget using a slider, switch between monthly and yearly pricing, and click on any plan to see its full details.

The website does not need any server or database. Everything runs directly in the browser. All the plan data is stored in JavaScript itself.

3. Problem Statement / Case Study Design

I planned the project before starting to code. I divided the whole website into 3 pages and made sure each page has its own HTML, CSS, and JS file so that the code stays clean and organized.

The 3 Pages

1. compare.html — This is the main page. It has a big table showing all 6 plans side by side. There are 3 filter controls at the top — a slider to filter by price, buttons to switch between monthly and yearly pricing, and a button to show or hide extra details.
2. all-plans.html — This page shows all 6 plans as cards. Each card shows the plan name, price, and key features. There is a button on each card to see full details.
3. plan-details.html — This page shows full details of one plan. The page does not have fixed content — JavaScript reads the plan name from the URL and fills everything on the page automatically.

File Structure

- compare.html + compare.css + compare.js
- all-plans.html + all-plans.css
- plan-details.html + plan-details.css + plan-details.js

Design of the Website

- I used a dark theme with a blue glow effect to make it look modern and professional.
- The fonts used are Syne for headings and DM Sans for normal text, loaded from Google Fonts.
- The table header stays fixed at the top even when you scroll down, so you always know which column belongs to which plan.
- The website works on both laptop and mobile screens because I used CSS Flexbox and Grid.

4. Methods & Algorithms / Technology Applied

Technologies I Used

- HTML5 — To create the structure of all 3 pages. I used tables, lists, buttons, input sliders, and links.
- CSS3 — To style the pages. I used dark background colors, glassmorphism effect (frosted glass look), sticky headers, hover effects, and animations.
- JavaScript — To add working features like the premium filter, pricing toggle, and dynamic plan details page.
- Google Fonts — For better looking text on the website.

How the Premium Filter Works

I stored all 6 plans in an array in JavaScript. Each plan has its monthly price and column position in the table. When the user moves the slider, the code checks each plan's price. If the price is higher than the slider value, it hides that column by setting display: none. Otherwise it shows it.

How the Monthly / Yearly Toggle Works

I stored both the monthly and yearly price directly in the HTML using custom attributes called data-monthly and data-yearly. When the user clicks Monthly or Yearly button, JavaScript reads these attributes and updates the text shown on screen. This way I did not need to calculate anything — the values were already stored.

How the Plan Details Page Works

This was the most interesting part. I stored all the data of all 6 plans in JavaScript as objects. When a user clicks "View" on any plan, the URL changes to plan-details.html?plan=smartshield (for example). When the details page opens, JavaScript reads the word "smartshield" from the URL, finds the matching plan object, and fills the page with that plan's information automatically.

I used split() to read the URL instead of URLSearchParams because split() works even when you open the files directly from your computer (file:// protocol).

5. Implementation Details and Snapshots

compare.html — The Comparison Table

This page has a sticky navigation bar at the top, a hero section with the page title, a filter section with 3 controls, and a comparison table with 6 plan columns. The table has 9 rows comparing different features like premium, coverage limit, hospitals, waiting period, dental cover, vision cover, maternity cover, mental health cover, and senior-friendly features.

compare.js — JavaScript Features

This file has 3 main functions:

- filterByPremium() — Hides or shows table columns based on slider value.
- switchPricing(mode) — Updates all price cells to show monthly or yearly prices.
- toggleCoverage() — Shows or hides the coverage detail rows in the table.

all-plans.html — Plan Cards Page

This page shows all 6 plans as individual cards using CSS Grid. Each card has the plan name, a colored badge (like "Popular", "Family", "Senior"), the monthly price, a list of 6 key features, and a "View Full Details" button. On hover, the cards lift up slightly with a smooth animation.

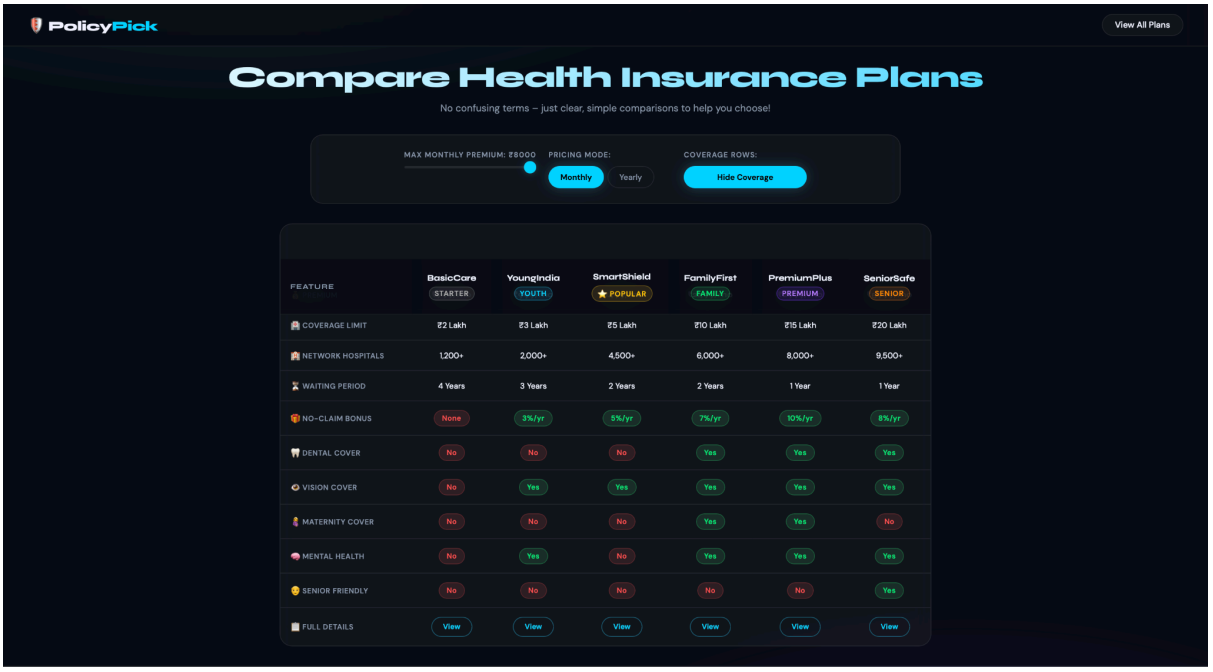
plan-details.html + plan-details.js — Dynamic Details Page

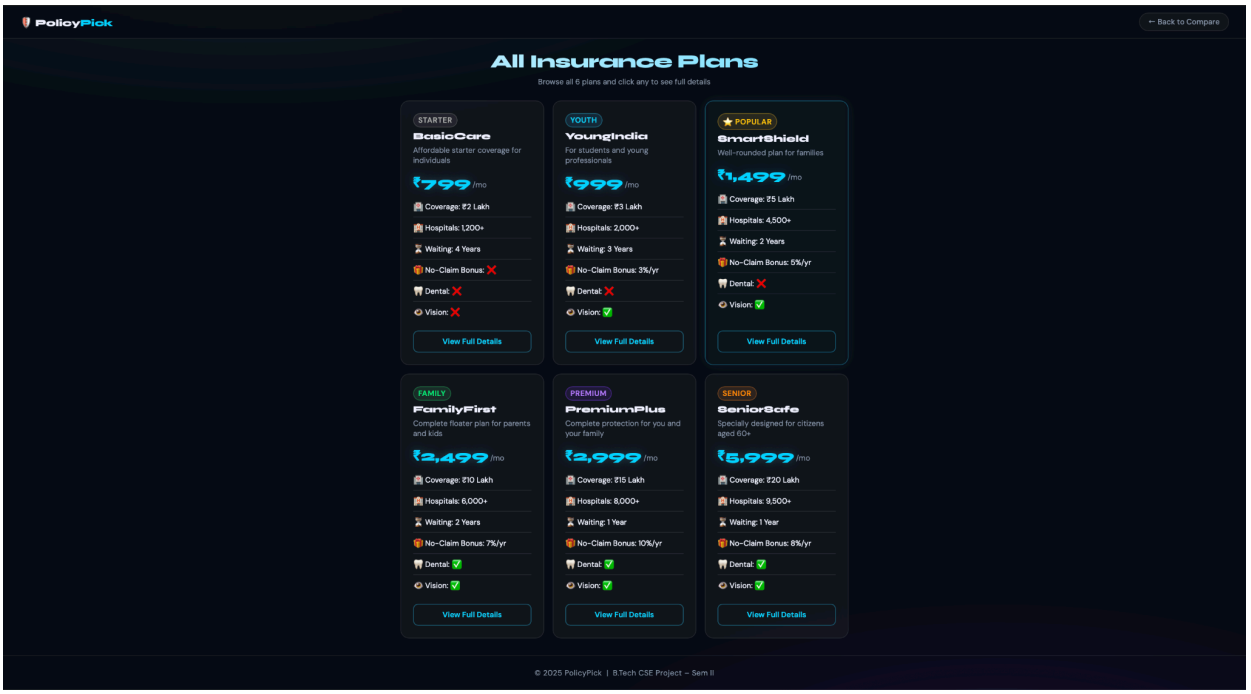
The HTML page has empty containers. The JavaScript file has all 6 plans stored as objects with details like name, coverage, hospitals, waiting period, what is covered, monthly price, yearly price, and no-claim bonus info. When the page loads, the script reads the URL, picks the right plan, and fills all the empty containers with that plan's data.

The 6 Insurance Plans in the Project

- BasicCare — Rs. 799/month, Rs. 2 Lakh coverage — for individuals on a low budget.
- YoungIndia — Rs. 999/month, Rs. 3 Lakh coverage — for students and young professionals.
- SmartShield — Rs. 1,499/month, Rs. 5 Lakh coverage — most popular, good for families.
- FamilyFirst — Rs. 2,499/month, Rs. 10 Lakh coverage — includes maternity and child vaccination.
- PremiumPlus — Rs. 2,999/month, Rs. 15 Lakh coverage — full coverage with all features.
- SeniorSafe — Rs. 5,999/month, Rs. 20 Lakh coverage — specially designed for people above 60.

Screen Shot of Pages -





6. Results and Conclusion

Results

After completing the project, I tested all 3 pages and all features are working correctly:

- The premium slider correctly hides and shows plan columns based on the selected budget.
- The Monthly and Yearly toggle correctly updates all prices on the comparison table.
- The coverage toggle correctly shows and hides the detail rows.

- Clicking "View" on any plan opens the details page with the correct information for that plan.
- The "View All Plans" button in the navbar correctly opens the all-plans page with all 6 cards.
- All 3 pages look good and the dark theme is consistent across the whole website.

What I Learned

- How to create and style HTML tables with sticky headers.
- How to use JavaScript to show and hide HTML elements dynamically.
- How to store data as JavaScript objects and use it to fill a page.
- How to pass information between pages using URL parameters.
- How to design a dark theme using CSS variables and glassmorphism effects.

Conclusion

PolicyPick is a complete front-end web project that solves a real problem — making health insurance comparison easy for everyone. I built the entire project using only HTML, CSS, and JavaScript without any backend or framework.

This project helped me understand how websites actually work, how JavaScript connects with HTML to create interactive features, and how to organize a multi-page web project properly. I am confident I can explain every part of this project because I have written all the code myself and understand what each line does.

7. References

1. W3Schools — HTML, CSS, JavaScript tutorials. https://www.w3schools.com
2. MDN Web Docs — Official HTML and JavaScript reference. https://developer.mozilla.org
3. Google Fonts — Syne and DM Sans fonts. https://fonts.google.com
4. CSS-Tricks — Guide to CSS Flexbox, Grid, and sticky positioning. https://css-tricks.com
5. JavaScript.info — JavaScript tutorial for beginners. https://javascript.info
6. ITM Skills University — B.Tech CSE Semester II Case Study guidelines.